# Stream Cipher Blind Time Long Code (SC-BLTC) Protocol Specification

## 1. System Parameters

The following global parameters are predefined:

- $F_s$ (Sampling Rate): 25 kHz. This is the time-domain resolution in simulation/implementation.
- $R_c$ (Chip Rate): 5 kcps. This results in a signal bandwidth of approximately 5–6 kHz (depending on the RRC roll-off factor $\alpha = 0.25$). $T_c = 1/R_c$.
- $OSF$ (Oversampling Factor): $OSF = F_s/R_c = 5$. $OSF$ is fixed as an integer.
- $SF$ (Spreading Factor): 1024. The processing gain per spread symbol is $G_p = 10\log_{10}(1024) \approx 30\,\text{dB}$.
- $k$ (Bits per Walsh Symbol): Fixed at $k = 8$.
- $M_W$ (Walsh Orthogonal Set Size): $M_W = 2^k = 256$ (256-ary orthogonal modulation).
- $R_{\text{sym}}$ (Spread Symbol Rate): $R_{\text{sym}} = R_c/SF = 5000/1024 \approx 4.88\,\text{sym/s}$.
- $R_{\text{bit}}$ (Post-spreading Bit Rate): $R_{\text{bit}} = k \cdot R_{\text{sym}} \approx 39.06\,\text{bps}$ (excluding preamble/pilot symbols and FEC overhead).
- $R_{\text{FEC}}$ (Polar Code Rate): Used for Forward Error Correction, fixed at 1/2. The information rate is $R_{\text{info}} = R_{\text{bit}} \times R_{\text{FEC}}$ (excluding Header/CRC/padding overhead and preamble/pilot symbol overhead).
- $K_{\text{sec}}$ (Shared Secret Key): A 256-bit key, pre-shared between the transmitter and receiver.
- $T_{\text{epoch}}$ (Time Epoch): Defined as the Unix Epoch (1970-01-01 00:00:00 UTC).
- $IV_{\text{res}}$ (Time Counter Resolution): Defines the time granularity for generating the spreading code seed, fixed at 1 ms.
- **Time Synchronization Assumption**: The Tx's local clock is roughly aligned with the Rx's time reference at the moment of transmission. Therefore, the Tx's $TI_{\text{tx}}$ is guaranteed to fall within the Rx's blind search window $[t_{\text{rx}} - W,\ t_{\text{rx}} + W]$.
- $M$ (FEC Codeword Length, coded bits): Fixed value $M = 512$.
- $K$ (FEC Information Bit Length, uncoded bits): Fixed value $K = 256$, satisfying $K = M \cdot R_{\text{FEC}}$.
- **FEC**: Uses an Arikan Polar code with $(N, K) = (512, 256)$ and CRC-aided SCL (CA-SCL) decoding.
- **CRC**: A CRC-32C is appended inside $U$ and is used both for post-decoding verification and for CA-SCL list selection.
- **Number of Preamble Symbols**: Fixed at $N_{\text{pre}} = 2$, used for acquisition and phase ambiguity resolution.
- **Number of Data Symbols**: $N_{\text{data}} = M/k = 64$.
- **Number of Pilot Symbols**: Fixed at $N_{\text{pilot}} = 16$. Pilots are used for continuous phase tracking under short coherence times of HF channels and to prevent frame-wide coherent demodulation failure due to cycle slips or phase flips.
- **Pilot Insertion Rule**: After the preamble, the $N_{\text{data}}$ data symbols are divided into $N_{\text{blk}} = 16$ blocks of 4 symbols each. A pilot symbol is inserted at the beginning of each block. The structure of a single block is: `Pilot + Data + Data + Data + Data`.
- **Number of Spread Symbols per Frame**: $N_{\text{sym}} = N_{\text{pre}} + N_{\text{pilot}} + N_{\text{data}} = 82$.
- $BW_{\text{hop}}$ (Hopping Bandwidth): 8 kHz. The hop range is centered at baseband 0 Hz and spans ±4 kHz.
- $N_{\text{bins}}$ (Frequency Bins): 256. Frequency resolution is $\Delta f = BW_{\text{hop}}/N_{\text{bins}} \approx 31.25\,\text{Hz}$.
- $L_{\text{sym}}$ (Spread Symbol Length): Fixed at $L_{\text{sym}} = SF = 1024$ chips for the information-bearing segment of each spread symbol.
- $L_{\text{jitter}}$ (Guard Interval / Jitter Range): A per-symbol guard/noise segment appended after each spread symbol, with
$$j_\ell \in [40, 90] \text{ chips}, \quad \ell = 0, \ldots, N_{\text{sym}} - 1.$$
This corresponds to 8 ms ∼ 18 ms at $R_c = 5$ kcps.
- $T_{\text{sym,avg}}$ (Average Symbol Duration): Approximately $1024 + 65 = 1089$ chips, i.e. ≈ 217.8 ms.
- $L_{\text{frame}}$ (Total Frame Length): Due to the per-symbol guard/noise insertion, the total frame length in chips is no longer a fixed constant. Instead,
$$L_{\text{frame}} = \sum_{\ell=0}^{N_{\text{sym}}-1} (L_{\text{sym}} + j_\ell),$$
while the symbol count remains fixed at $N_{\text{sym}} = 82$. The number of samples is $L_{\text{frame}} \cdot OSF$ (plus any optional tail padding used for filter decay).
- $E_b/N_0$ **Benchmark**: For a fixed $SF = 1024$, using $k = 8$ reduces the "number of chips allocated per bit", resulting in a $10\log_{10}(k) = 9\,\text{dB}$ processing gain loss. The receiver employs $M_W$-way orthogonal Walsh

matched filtering, which provides a 3 dB equivalent advantage over binary decisions at the same $E_b/N_0$. The net change is a 6 dB degradation.

- **Walsh Code Definition**: Uses the row vectors $W_m[j] \in \{+1, -1\}$ of the order-$SF = 1024$ Sylvester-Hadamard matrix, where

$$W_m[j] = (-1)^{\text{popcount}(m \& j) \bmod 2}, \quad m \in [0, 1023],\ j \in [0, 1023]$$

This protocol only uses the first $M_W$ orthogonal rows for $m \in [0, 255]$. $W_0[j] \equiv +1$.

## 2. Core Primitive: Cryptographically Secure Spreading Sequence Generator (CSPRNG Spreading)

AES-CTR mode is used as the Pseudo-Random Number Generator (PRNG).

#### Generation Function `GenCode(Key, TimeIndex, N_sym)`

**Inputs**:

- *Key*: The shared secret key.
- *TimeIndex*: An integer time counter for the transmission instant.
- $N_{\text{sym}}$: The number of spread symbols per frame (fixed at 82).

**Outputs**:

1. Cryptographic chip masks $C_{\text{seq}}$ for de-spreading, organized as $N_{\text{sym}}$ blocks of length $SF = 1024$ chips.
2. Frequency control sequence $F_{\text{seq}} = \{f_0, f_1, \ldots, f_{N_{\text{sym}}-1}\}$ (Hz).
3. Jitter/guard control sequence $J_{\text{seq}} = \{j_0, j_1, \ldots, j_{N_{\text{sym}}-1}\}$ (chips).

**Process**:

1. **Construct Nonce/Counter**: Follows the standard CTR "Nonce + Counter" structure.
   - The 96-bit Nonce is defined as $\mathit{Nonce} = (\mathit{TimeIndex} \parallel \mathit{Domain})$, where *TimeIndex* is 64-bit (endianness must be fixed in implementation), and *Domain* is a 32-bit constant `0x424C5443` (ASCII for "BLTC"), used for domain separation to prevent keystream reuse across different applications.
   - A 32-bit BlockCounter starts from 0 and increments for each block, forming the AES block input $IV_{\text{block}} = (\mathit{Nonce} \parallel \mathit{BlockCounter})$.
2. **AES-CTR Generation**: Encrypt an all-zero data stream to produce the keystream bytes.
3. **Structured Consumption (Per Symbol)**: For the $\ell$-th spread symbol ($\ell = 0, \ldots, N_{\text{sym}} - 1$), consume the keystream as:
   1. **Data Block**: Take the next 1024 bits and map them (MSB-first per byte) to the bipolar chip mask $C_{\text{seq}}[\ell][j] \in \{+1, -1\}$ using `0 -> +1, 1 -> -1`.
   2. **Control Block**: Take the next 16 bits as metadata:
      - High 8 bits: $V_{\text{freq}} \in [0, 255]$
      - Low 8 bits: $V_{\text{jitter}} \in [0, 255]$
   3. **Parameter Mapping**:
      - Hop offset:
      $$f_\ell = \left(\frac{V_{\text{freq}}}{256.0} - 0.5\right) \cdot BW_{\text{hop}} \quad \text{(Hz)}.$$
      - Guard length:
      $$j_\ell = 40 + \left\lfloor \left(\frac{V_{\text{jitter}}}{256.0}\right) \cdot 50 \right\rfloor \quad \text{(chips)},$$
      which yields $j_\ell \in [40, 90]$.
4. **Endianness and Bit Order Convention**:

- `TimeIndex` is encoded as **64-bit big-endian**.
- `Domain` and `BlockCounter` are encoded as **32-bit big-endian**.
- The AES-CTR output bytes are expanded into a bitstream with **bitorder=big** (MSB-first for each byte).

5. **Non-Reuse Constraint**: It is strictly forbidden to transmit twice with the same *TimeIndex* under the same shared key $K_{\text{sec}}$, as this would reuse the same CTR keystream. The transmitter powers down after each transmission and ensures the time interval between two consecutive transmissions is no less than $IV_{\text{res}}$, thus guaranteeing the uniqueness of *TimeIndex*.

## 3. Transmitter Protocol

The transmitter is standalone; it does not query network time and relies solely on its local crystal oscillator.

> **Note**: The transmitter does not need network time synchronization, but the system assumes its local clock is roughly aligned with the receiver's time reference at the moment of transmission, as detailed in Section 1.

#### Step A0: Frame Format

To ensure the receiver can pre-determine the symbol schedule and make decoding decisions, this protocol uses a fixed **symbol count** per frame ($N_{\text{sym}} = 82$). Due to the per-symbol guard/noise insertion, the **total sample length** of a frame is not a fixed constant, but it is fully determined once *TimeIndex* is guessed (through the generated $J_{\text{seq}}$).

- **Information Bits (uncoded)**: $U$, with a length of $K = 256$ bits, composed of the following parts:
  1. **Header (plaintext/structural fields)**:
     - `Ver`: Protocol Version (4 bits).
     - `Type`: Message Type (4 bits).
     - `Len`: Payload Length (8 bits, in Bytes).
  2. **Payload**: Service data bits.
  3. **CRC**: A CRC-32C is computed over the Header+Payload for post-decoding verification.
  4. **Padding**: If Header+Payload+CRC does not fill up $K$ bits, it is padded with zeros.
- **Polar Encoding**: $U$ is encoded with a fixed code rate $R_{\text{FEC}} = 1/2$ into a codeword $B$ of length $M = 512$ (coded bits) using a fixed Polar code $(N, K) = (512, 256)$.

  **Frozen Set and Bit Mapping**:
  1. Construct a length-$N$ vector $\mathbf{u} = \{u_0, \ldots, u_{N-1}\}$ with a fixed frozen set $\mathcal{F}$ of size $N - K$. For all $i \in \mathcal{F}$, set $u_i = 0$. For all $i \notin \mathcal{F}$ (the information set), fill $u_i$ with the next bit from $U$ in increasing index order.
  2. The frozen set $\mathcal{F}$ is determined using Polarization Weight (PW) ranking for $N = 512$ with $\beta = 2^{1/4}$:

  $$w(i) = \sum_{j=0}^{\log_2 N - 1} b_j(i)\, \beta^j, \quad b_j(i) \in \{0, 1\} \text{ is bit } j \text{ of } i.$$

  The $K$ indices with the largest $w(i)$ form the information set; all other indices are frozen.
  3. Apply the standard Arikan Polar transform to obtain the codeword $\mathbf{B} = \{b_0, \ldots, b_{N-1}\}$.
- **Full-Frame Interleaving**: Apply a fixed bit permutation $\pi(\cdot)$ over the full codeword to produce an interleaved sequence $B^{(\pi)} = \{b_0^{(\pi)}, \ldots, b_{M-1}^{(\pi)}\}$:

  $$b_j^{(\pi)} = b_{\pi(j)}, \quad \pi(j) = (A \cdot j + B) \bmod M, \quad A = 109,\ B = 37.$$

  This is a bijection because $M = 2^9$ and $A$ is odd.
- **Symbol Mapping (256-ary Walsh)**: $B^{(\pi)}$ is grouped into $N_{\text{data}} = 64$ symbol indices $m_q \in [0, 255]$ of $k = 8$ bits each (MSB first):

$$m_q = \sum_{t=0}^{k-1} b_{qk+t}^{(\pi)} \cdot 2^{k-1-t}, \quad q = 0, \ldots, N_{\text{data}} - 1$$

These $N_{\text{data}}$ indices are mapped to $N_{\text{data}}$ orthogonal Walsh spread symbols.

- **Symbol Arrangement (Preamble + Pilots + Data)**:
  - Spread symbols are numbered $\ell = 0, \ldots, N_{\text{sym}} - 1$.
  - $\ell = 0$ and $\ell = 1$ are the two Preamble symbols.
  - Following the preamble are $N_{\text{blk}} = 16$ blocks, numbered $r = 0, \ldots, 15$. Each block consists of 5 spread symbols: 1 pilot + 4 data.
  - The position of the pilot symbol for block $r$ is

$$\ell_{\text{pilot}}(r) = 2 + 5r$$

  - The position of the $q$-th data symbol ($q = 0, \ldots, 63$) is determined as follows: let $r = \lfloor q/4 \rfloor$ and $s = q \bmod 4$, then

$$\ell_{\text{data}}(q) = 2 + 5r + 1 + s$$

**Step A: Payload Preparation**

Assemble the frame according to Step A0 to get the fixed-length information bits $U$ (including CRC and padding). Perform Polar encoding on $U$ to get the codeword bit sequence $B = \{b_0, \ldots, b_{M-1}\}$. Apply the full-frame interleaver to get $B^{(\pi)}$, and pack it into a sequence of symbol indices $\{m_0, \ldots, m_{N_{\text{data}}-1}\}$ using $k = 8$ bits per symbol.

**Step B: Determine Transmission Time Seed**

Read the current local time $t_{\text{tx}}$. Calculate the time counter:

$$TI_{\text{tx}} = \left\lfloor \frac{t_{\text{tx}}}{IV_{\text{res}}} \right\rfloor$$

**Note**: This $TI_{\text{tx}}$ is implicitly embedded in the signal, and the receiver must guess this value.

**Step C: Generate Full-Frame Spreading Code**

Call the generation function:

$$(C_{\text{seq}}, F_{\text{seq}}, J_{\text{seq}}) = \texttt{GenCode}(K_{\text{sec}}, TI_{\text{tx}}, N_{\text{sym}})$$

where: - $C_{\text{seq}}$ provides the $N_{\text{sym}}$ per-symbol chip masks (each of length $SF = 1024$). - $F_{\text{seq}}$ provides the per-symbol hop offsets $f_\ell$ (Hz). - $J_{\text{seq}}$ provides the per-symbol guard lengths $j_\ell$ (chips).

**Step D: Walsh Orthogonal Spreading Modulation**

The cryptographic chip sequence is used as a "mask/scrambling code" and is combined with the orthogonal Walsh rows to generate the transmitted chips for each spread symbol.

The chips for the $\ell$-th spread symbol ($\ell = 0, \ldots, N_{\text{sym}} - 1$) are denoted as $S[\ell \cdot SF + j]$, where $0 \le j < SF$.

- **Preamble Symbols** ($\ell = 0, 1$): Fixed to use a Barker-2 synchronization word with Walsh index 0, i.e., $[+W_0, -W_0]$:
  - **Preamble symbol 0** ($\ell = 0$, $+W_0$):

$$S[0 \cdot SF + j] = C_{\text{seq}}[0 \cdot SF + j]$$

– **Preamble symbol 1** ($\ell = 1$, $-W_0$):

$$S[1 \cdot SF + j] = -C_{\text{seq}}[1 \cdot SF + j]$$

These two preamble symbols are used for acquisition and phase ambiguity resolution and do not carry information bits.

- **Pilot Symbols** ($\ell = \ell_{\text{pilot}}(r)$, $r = 0, \ldots, 15$): Fixed to use $W_0[j] \equiv +1$, therefore

$$S[\ell \cdot SF + j] = C_{\text{seq}}[\ell \cdot SF + j]$$

Pilot symbols are used for continuous phase tracking and cycle slip suppression and do not carry information bits.

- **Data Symbols** ($\ell = \ell_{\text{data}}(q)$, $q = 0, \ldots, N_{\text{data}} - 1$): Uses the $m_q$-th Walsh row. Let $\ell = \ell_{\text{data}}(q)$:

$$S[\ell \cdot SF + j] = W_{m_q}[j] \cdot C_{\text{seq}}[\ell \cdot SF + j]$$

This defines the information-bearing chip segment of each spread symbol (length $SF = 1024$ chips). The transmitted chip stream is formed by appending a per-symbol guard/noise segment after each spread symbol (Step D1.5), resulting in a variable-length chip stream.

#### Step D1.5: Guard Interval Insertion and Noise Filling

To mitigate multipath interference between hops and to conceal symbol boundaries, after the $\ell$-th spread symbol's 1024 data chips, append a length-$j_\ell$ guard/noise segment:

1. **Data Segment**: Send the $SF = 1024$ chips defined in Step D.
2. **Guard/Noise Segment**: Immediately follow with $j_\ell$ chips of Gaussian white noise $S_{\text{noise}}$.
    - The noise power is set to approximately **-3 dB** relative to the data-chip power.
    - This segment is not Walsh-spread and carries no information.

#### Step D2: Pulse Shaping and Oversampling

To constrain bandwidth and improve multipath resistance, the baseband chip sequence must be pulse-shaped.

- **Shaping Filter**: A Root-Raised-Cosine (RRC) filter $g_{\text{rrc}}(t)$ is used.
- **Oversampling**: The shaped waveform is output at a sampling rate of $F_s = OSF \cdot R_c$.

Let $S_{\text{base}}[n]$ be the RRC-shaped complex baseband waveform at the sampling rate, generated from the chip stream that includes both the data segments and the guard/noise segments. Its length is $L_{\text{frame}} \cdot OSF$ (plus any optional tail padding used for filter decay).

> **Note**: RRC pulse shaping is applied **before** digital frequency rotation (Step D3). During guard/noise segments, it is recommended to apply a short **soft ramp** at the guard boundaries to avoid sharp envelope edges.

**Step D3: Digital Frequency Rotation and Phase Continuity**

To realize hopping while keeping extremely low out-of-band radiation, hopping is performed by complex rotation at baseband with a **cumulative** phase.

Let $f_\ell$ be the hop offset for symbol $\ell$ (from $F_{\text{seq}}$). Define a cumulative phase $\Phi[n]$ and a per-sample phase increment:

$$\Delta\omega_\ell = 2\pi \frac{f_\ell}{F_s}.$$

For samples $n$ that fall within the $\ell$-th symbol's **time span** (including its guard/noise segment), update:

$$\Phi[n] = \Phi[n-1] + \Delta\omega_\ell.$$

Transmit waveform:

$$S_{\text{tx}}[n] = S_{\text{base}}[n] \cdot e^{j\Phi[n]}.$$

> **Important (Phase Continuity)**: $\Phi[n]$ must remain cumulative across symbol boundaries (do not reset at symbol transitions). This ensures smooth time-domain transitions at hop changes and strongly suppresses spectral splatter.

**Step E: Transmission**

Up-convert $S_{\text{tx}}$ and transmit.

**Step E2: Soft Shutdown**

1. **Tail Padding**: After transmitting the $N_{\text{sym}}$ symbols, append an additional $N_{\text{tail}} = 8$ all-zero symbols (Zero Padding) to allow the RRC filter's impulse response to decay naturally.
2. **Power Ramp-Down**: For the last $L_{\text{ramp}}$ samples (recommended duration of about 20 ms), multiply the transmit waveform $S_{\text{tx}}[n]$ by a smooth-decaying window function $w[n]$ (from 1.0 down to 0.0) to prevent spectral splatter.
3. **Physical Shutdown**: After the digital waveform output has returned to zero, turn off the Power Amplifier (PA) in sequence, then turn off the Local Oscillator (LO) and system power after a 10 ms delay to avoid frequency drift artifacts.

## 4. Receiver Protocol

The receiver does not know $TI_{\text{tx}}$; it only knows that the transmission occurred sometime in the recent past. Assume the local time is $t_{\text{rx}}$, and the maximum clock drift and uncertainty window is $W = \pm 5$ seconds.

**Step A: Signal Buffering**

The receiver continuously records the signal into a circular buffer `Buffer`.

**Step B: Blind Acquisition (De-hopping + FFT)**

Because both the hop schedule $F_{\text{seq}}$ and the guard schedule $J_{\text{seq}}$ depend on *TimeIndex*, acquisition is performed under a *TI* hypothesis and explicitly reconstructs the corresponding time-frequency structure.

> The receiver first down-converts the signal to complex baseband I/Q. The acquisition stage operates directly on these **raw baseband samples**.

### 4.B.1 Search Space Definition

- **Time Hypothesis**: Iterate through all possible seeds $TI_{\text{search}} \in \left[\left\lfloor \frac{t_{\text{rx}}-W}{IV_{\text{res}}} \right\rfloor, \left\lfloor \frac{t_{\text{rx}}+W}{IV_{\text{res}}} \right\rfloor\right]$.
- **Intra-Epoch Starting Offset (Sample-Level)**: Search an unknown intra-epoch start offset at sample resolution over one IV: define $N_{\text{IV,samp}} = \text{round}(F_s \cdot IV_{\text{res}})$ and search $n_{\text{offset,total}} \in [0, N_{\text{IV,samp}} - 1]$.
- **Residual Frequency Offset Search**: After de-hopping (4.B.2), use an FFT to search residual CFO within a limited band $|\Delta f| \le f_{\text{search}}$ (e.g., 8 kHz).

### 4.B.2 Algorithm Outline

For each $TI_{\text{search}}$:

1. **Generate Schedules**:

$$(C_{\text{seq}}, F_{\text{seq}}, J_{\text{seq}}) = \texttt{GenCode}(K_{\text{sec}}, TI_{\text{search}}, N_{\text{sym}}).$$

2. **Compute Symbol Start Positions**: Using $J_{\text{seq}}$, compute the symbol start positions (in samples) relative to the frame start:

$$n_\ell = \left(\sum_{k=0}^{\ell-1}(SF + j_k)\right) \cdot OSF.$$

3. **De-hopping Rotation (Time-Frequency Grid Reconstruction)**: Construct a de-hopping phasor whose instantaneous frequency follows the hop sequence and remains phase-continuous:

$$D[n] = e^{-j\Phi[n]}, \quad \Phi[n] = \Phi[n-1] + 2\pi\frac{f_\ell}{F_s},$$

for samples $n$ within symbol $\ell$'s time span (data + guard/noise).

4. **Preamble FFT Detection**: For each start-offset hypothesis $n_{\text{offset,total}}$, extract a window covering **both** preamble symbols' **data segments** and coherently accumulate them for the coarse FFT score. Concretely, form a sparse window

$$y[n] = \begin{cases} x[n] \cdot s_0^*[n] & n \in [0, SF \cdot OSF) \\ 0 & n \in [SF \cdot OSF, (SF + j_0) \cdot OSF) \\ x[n] \cdot s_1^*[n - (SF + j_0) \cdot OSF] & n \in [(SF + j_0) \cdot OSF, (2SF + j_0) \cdot OSF) \end{cases}$$

where $s_0, s_1$ are the locally generated (hopped) TX-shaped preamble references for symbol 0 and 1 (with the Barker-2 sign pattern $+W_0, -W_0$) and the hop phasor is **phase-continuous** across the symbol boundary (i.e., $s_1$ includes the accumulated hop phase at its start). The guard/noise interval of length $j_0$ is ignored by inserting zeros. Zero-pad $y[n]$ to $N_{\text{FFT}}$ and take an FFT to find the residual CFO peak and produce a coarse detection score.

5. **Verification Using Pilots**: For top candidates, refine CFO on a small grid and verify using noncoherent pilot energy. Pilot symbol positions are computed using $n_\ell$, and the receiver integrates only over each pilot's 1024-chip data segment (ignoring the guard/noise segment).

6. **Timing Refinement and RAKE Initialization**: Refine the start offset by coherent correlation around the coarse estimate. For better timing discrimination under multipath, the receiver jointly uses **both** preamble symbols, taking into account the guard length $j_0$ between them.

   Concretely, for each candidate start offset $\hat{n}$ in the search window (around the coarse estimate), form the two complex correlations

$$z_0(\hat{n}) = \sum_{n=0}^{SF \cdot OSF - 1} x[\hat{n} + n] \cdot s_0^*[n],$$

$$z_1(\hat{n}) = \sum_{n=0}^{SF \cdot OSF - 1} x[\hat{n} + (SF + j_0) \cdot OSF + n] \cdot s_1^*[n],$$

where $s_0, s_1$ are the locally generated TX-shaped preamble references (with the Barker-2 sign pattern and phase-continuous hopping, as in Step 4).

The timing score is the **noncoherent** sum of the two per-symbol correlation energies:

$$e_0(\hat{n}) = |z_0(\hat{n})|^2, \quad e_1(\hat{n}) = |z_1(\hat{n})|^2, \quad \Lambda(\hat{n}) = e_0(\hat{n}) + e_1(\hat{n}).$$

Select the refined main-path start:

$$\hat{n}_0 = \arg\max_{\hat{n}} \Lambda(\hat{n}) \quad \text{(tie-break: smaller } \hat{n}\text{)}.$$

The receiver constructs the returned RAKE finger list $\{\hat{n}_{\text{finger},i}\}$ directly from these hypotheses:

1. Sort all hypotheses by descending $\Lambda(\hat{n})$ (tie-break: smaller $\hat{n}$).
2. Initialize the finger list with $\hat{n}_{\text{finger},0} = \hat{n}_0$.
3. Iterate the remaining hypotheses in sorted order and append $\hat{n}$ to the finger list iff:
   - $\hat{n} \geq \hat{n}_0$ (non-negative delay relative to the main path), and
   - $|\hat{n} - \hat{n}_{\text{finger},k}| \geq OSF$ for all already-selected fingers $k$ (minimum separation).
4. Stop when either:
   - the list length reaches $N_{\text{finger,out}} = \text{clamp}(N_{\text{finger,req}}, 1, 16)$, or
   - the hypothesis list is exhausted.

The acquisition output parameters are: $\widehat{TI}_{\text{tx}}$, the main path starting position $\hat{n}_0$, the refined CFO estimate $\widehat{\Delta f}$, an **ordered list** of RAKE finger starting positions $\{\hat{n}_{\text{finger},i}\}_{i=0}^{N_{\text{finger,out}}-1}$ (with $\hat{n}_{\text{finger},0} = \hat{n}_0$), and the verification statistic $p_{\max}$.

### Step C: RAKE Reception & Carrier Synchronization

After successful acquisition, the system switches to tracking and demodulation mode.

Using the acquisition outputs $(\widehat{TI}_{\text{tx}}, \hat{n}_0, \widehat{\Delta f}, \{\hat{n}_{\text{finger},i}\})$, perform the following preprocessing on the **raw** baseband samples:

1. **Post-Acquisition Sample-Offset Refinement**: Refine the sample start position by searching a small integer window $\Delta n \in [-6, 6]$ around $\hat{n}_0$ (and apply the same $\Delta n$ to all $\{\hat{n}_{\text{finger},i}\}$). This step is required because with phase-continuous hopping (Step D3), a sample-index error $\Delta n$ introduces a hop-dependent phase error approximately

   $$\Delta\phi_\ell \approx 2\pi \frac{f_\ell}{F_s} \Delta n,$$

   which can significantly reduce coherent combining/LLR quality at low SNR. The receiver evaluates the refinement candidates in the fixed order:

   $$\Delta n \in \{0, -1, +1, -2, +2, \ldots, -6, +6\}.$$

   For each candidate $\Delta n$, the receiver applies the same shift to every finger offset $\{\hat{n}_{\text{finger},i}\}$, then performs CFO derotation, de-hopping, matched filtering, and a full-frame demod/decode.

   - If any $\Delta n$ yields a CRC-valid decode, the receiver selects the **first** such $\Delta n$ in the order above and returns that decode result.

- If no $\Delta n$ yields a CRC-valid decode, the receiver selects the $\Delta n$ that maximizes the verification statistic.

$$V(\Delta n) = V_{\text{pre}}(\Delta n) + V_{\text{pilot}}(\Delta n),$$

where

$$V_{\text{pre}}(\Delta n) = \sum_i \left| \left( \sum_{j=0}^{SF-1} \mathbf{u}_{i,0}[j] \right) - \left( \sum_{j=0}^{SF-1} \mathbf{u}_{i,1}[j] \right) \right|^2,$$

$$V_{\text{pilot}}(\Delta n) = \sum_{\ell \in \mathcal{P}} |z_\ell|^2,$$

$\mathcal{P}$ is the set of pilot symbol indices, and $z_\ell$ is the PLL prompt sum used in Step C.2(a) for preamble/pilots.

2. Apply an initial CFO derotation using $\widehat{\Delta f}$.
3. Reconstruct the hop/guard schedule from $\widehat{TI}_{\text{tx}}$ (via `GenCode`) and apply a global phase-continuous **dehopping** rotation so that the information-bearing parts of the frame are moved back near 0 Hz.
4. Apply the receive RRC matched filter to form the matched-filtered sequence.

All subsequent RAKE/DLL/PLL processing in Step C/D operates on this matched-filtered, de-hopped signal.

**0. Full-Frame Local Code Generation**

Use the acquired $\widehat{TI}_{\text{tx}}$ to call

$$(C_{\text{seq}}, F_{\text{seq}}, J_{\text{seq}}) = \texttt{GenCode}(K_{\text{sec}}, \widehat{TI}_{\text{tx}}, N_{\text{sym}})$$

to generate the full-frame cryptographic chip masks (for de-masking) as well as the hop/guard schedules (for de-hopping and symbol windowing).

**1. RAKE Structure**

The acquisition stage (4.B.2–4.B.3) outputs an ordered list of RAKE finger starting positions $\{\hat{n}_{\text{finger},i}\}_{i=0}^{N_{\text{finger,out}}-1}$ (in samples), with $\hat{n}_{\text{finger},0} = \hat{n}_0$. The list is ordered by descending preamble correlation energy (tie-break: smaller offset).

Use at most three fingers for tracking and MRC combining:

$$N_{\text{finger}} = \min(3, N_{\text{finger,out}}).$$

Define the per-finger integer delay (samples) relative to the main path:

$$\Delta d_i = \hat{n}_{\text{finger},i} - \hat{n}_{\text{finger},0}.$$

All RAKE/DLL/PLL/MRC processing in Step C uses only the first $N_{\text{finger}}$ offsets and the corresponding $\Delta d_i$.

**Window Gating (Data vs Guard/Noise)**

For each spread symbol $\ell$, the receiver knows: - the **data** segment length is fixed at 1024 chips, - the following **guard/noise** segment length is $j_\ell$ chips (from $J_{\text{seq}}$).

The RAKE receiver must integrate and search multipath only over the 1024-chip data window. During the $j_\ell$ guard/noise interval, the RAKE must freeze (stop integrating and avoid updating its correlators) to prevent integrating the transmitter-inserted noise and to allow multipath energy from the previous hop to decay.

**2. Dual-Loop Tracking**

Due to the frame duration being on the order of seconds and the transmitter's unstable clock, both carrier frequency drift and chip clock drift must be overcome simultaneously.

- **a. Carrier Tracking (PLL/Costas Loop)**: Run the PLL on the **de-hopped** baseband signal to track only the residual frequency/phase error (e.g., oscillator drift and Doppler). Initialize with the coarse frequency offset $\widehat{\Delta f}$ from acquisition.

  – **Implementation Requirement**: A small closed-loop PLL/Costas loop must be run at the symbol rate and updated on every spread symbol's **data segment** (preamble, pilots, and data). The loop must not update during the guard/noise intervals $j_\ell$.

  Define the symbol-rate NCO state with phase $\theta_\ell$ (rad) and phase increment $\omega_\ell$ (rad/symbol), updated using a second-order Type-II loop:

$$e_\ell = \mathrm{atan2}(\Im\{z_\ell\}, \Re\{z_\ell\}), \quad \omega_{\ell+1} = \omega_\ell + K_i e_\ell, \quad \theta_{\ell+1} = \theta_\ell + \omega_{\ell+1} + K_p e_\ell.$$

  The input to the error detector, $z_\ell$, is chosen as:

  – **Preamble/Pilot (known $W_0$)**: $z_\ell = \sum_{j=0}^{SF-1} \mathbf{u}_\ell[j]$;
  – **Data Symbol (decision-directed)**: First, make a Walsh decision $\hat{m}_\ell = \arg\max_{m\in[0,255]} |R_\ell[m]|$, then use $z_\ell = R_\ell[\hat{m}_\ell]$.

  **Symbol-Rate Frequency-Hypothesis Bank**

  Because $T_{\mathrm{sym}}$ can be large (e.g., $\approx 205\,\mathrm{ms}$), even sub-Hz residual Doppler causes significant phase rotation within a single symbol. In low SNR, a pure phase-detector PLL can therefore suffer cycle slips. A small brute-force frequency bank around the current PLL estimate at the symbol rate is used to avoid this issue.

  Let $f_{\mathrm{PLL},\ell} = \omega_\ell/(2\pi\, T_{\mathrm{sym}})$ (Hz). For each symbol $\ell$:

  1. **Hypotheses**: generate $f_k = f_{\mathrm{PLL},\ell} + \Delta f_k$ with $\Delta f_k \in \{-4, -3.75, \ldots, +4\}$ Hz (step 0.25 Hz), and map to $\omega_k = 2\pi f_k\, T_{\mathrm{data}}$, where $T_{\mathrm{data}} = SF/R_c$ is the fixed data-segment duration.
  2. **Per-hypothesis demod**:
     – **Preamble/Pilot ($W_0$ known)**: select $k_{\mathrm{best}} = \arg\max_k \left|\sum_{j=0}^{SF-1} \mathbf{u}_{\ell,k}[j]\right|$.
     – **Data**: for each $k$, compute $R_{\ell,k}[m]$ via a 1024-point FHT and select $(k_{\mathrm{best}}, \hat{m}_\ell) = \arg\max_{k,m\in[0,255]} |R_{\ell,k}[m]|$.
  3. **PLL update**: always advance $\theta$ by the selected $\omega_{\ell,k_{\mathrm{best}}}$, then apply the phase-detector correction using $z_\ell$ (pilot/preamble prompt sum or $R_{\ell,k_{\mathrm{best}}}[\hat{m}_\ell]$).
  4. **Frequency snap (anti-slip)**: if the frequency-bank confidence is at least 0.15 and the same non-zero $\Delta f_{k_{\mathrm{best}}}$ is selected (within $\pm 0.125$ Hz) for 3 consecutive symbols, then:
     – if $|\Delta f_{k_{\mathrm{best}}}| \geq 0.75$ Hz, update $\omega_\ell \leftarrow \mathrm{clamp}(\omega_\ell + 2\pi\, \Delta f_{k_{\mathrm{best}}}\, T_{\mathrm{data}}, [-\omega_{\max}, +\omega_{\max}])$,
     – otherwise do not change $\omega_\ell$, and reset the snap state. This implementation uses $\omega_{\max} = 2\pi \cdot 200\,\mathrm{Hz} \cdot T_{\mathrm{data}}$.

  **Loop Gain Design (Discrete-Time)**

  The loop gains $(K_p, K_i)$ should be designed in discrete time (as a function of the update period $T_{\mathrm{sym}}$). Small-$T$ analog approximations like $K_p \approx 2\zeta\omega_n T$ and $K_i \approx (\omega_n T)^2$ should not be used when $\omega_n T$ is not small.

  A standard choice is to specify loop bandwidth $B_L$ (Hz) and damping factor $\zeta$, set $\omega_n = 2\pi B_L$, and map the continuous-time poles to discrete time via $z = \exp(sT)$:

$$r = \exp(-\zeta\omega_n T), \quad \phi = \omega_n\sqrt{1-\zeta^2}\,T, \quad z_{1,2} = re^{\pm j\phi}.$$

  For the update equations above, the corresponding stable discrete-time gains are:

$$K_p = 1 - r^2, \quad K_i = 1 + r^2 - 2r\cos\phi.$$

- **b. Code Timing Tracking (DLL - Delay Locked Loop)**: Use an Early-Late Gate to track the expansion/contraction of the chip clock (Code Doppler). Due to the transmitter's crystal oscillator drift, a cumulative drift of several chips can occur over a transmission of more than ten seconds. Failure to track this will lead to a mismatch of the spreading sequence. The DLL dynamically adjusts the sampling instant to keep the correlation peak in the "Prompt" channel.

**3. Despreading and Combining**

For each spread symbol $\ell$ and each finger $i$, perform the following operations and combine in the chip domain:

1. Starting from $\hat{n}_{\text{finger},i}$, extract the received segment $Y_{i,\ell}$ for this symbol.
2. Compensate for the carrier phase estimated by the PLL/Costas loop: $Y'_{i,\ell}(t) = Y_{i,\ell}(t) \cdot e^{-j\theta_\ell}$.
3. After matched filtering and DLL alignment, extract $SF$ chip samples at the chip instants to get the chip vector $\mathbf{y}_{i,\ell}[j]$.
4. De-mask using the locally generated cryptographic chip mask: $\mathbf{u}_{i,\ell}[j] = \mathbf{y}_{i,\ell}[j] \cdot C_{\text{seq}}[\ell \cdot SF + j]$.
5. **Hop-delay phase compensation**: for the $i$-th finger, let $\Delta d_i$ be the integer delay (samples) relative to the main finger (Step C.1). For symbol $\ell$, compute

$$c_{i,\ell} = \exp\left(+j2\pi f_\ell \frac{\Delta d_i}{F_s}\right),$$

and apply

$$\mathbf{u}_{i,\ell}[j] \leftarrow \mathbf{u}_{i,\ell}[j] \cdot c_{i,\ell}.$$

The receiver applies this compensation to all branches that enter MRC, including prompt and the DLL Early/Late branches.

All fingers are combined using Maximal-Ratio Combining (MRC) by a weighted sum in the chip domain to obtain the de-masked chip vector for that symbol:

$$\mathbf{u}_\ell[j] = \sum_i w_i \cdot \mathbf{u}_{i,\ell}[j], \quad w_i = \frac{A_i^*}{\sum_k |A_k|^2}$$

where $A_i$ is the estimated complex channel gain for the $i$-th finger. $A_i$ is initialized by coherent averaging across the two preamble symbols ($\ell = 0, 1$) and is updated on every pilot symbol with an exponential smoother with $\alpha = \frac{1}{16}$:

$$\tilde{A}_i^{(r)} = \frac{1}{SF} \sum_{j=0}^{SF-1} \mathbf{u}_{i,\ell_{\text{pilot}}(r)}[j] \cdot e^{-j\theta_{\ell_{\text{pilot}}(r)}}, \quad A_i^{(r)} = (1-\alpha)A_i^{(r-1)} + \alpha\tilde{A}_i^{(r)}, \quad \alpha = \frac{1}{16}.$$

**Implementation Note**:

- **Fractional Sampling**: When the DLL/code phase requires sub-sample level adjustment, linear interpolation can be performed on the sample sequence after the matched filter to obtain the equivalent sample value at the chip instant.

**Step D: Walsh Demodulation and Polar (CA-SCL) Decoding**

**1. Pilot-aided Carrier Tracking**

The coherence time of HF channels is often significantly shorter than the frame duration. Therefore, this protocol periodically inserts pilot symbols within the frame, which are used to aid the closed-loop PLL/Costas in Step C to continuously track the carrier phase/residual frequency offset. In the intervals of 4 data symbols between two pilot blocks, the loop continues to update in a decision-directed manner to avoid intra-block loss of coherence.

**2. Walsh Matched Filtering (FHT)**

For each data symbol $\ell = \ell_{\text{data}}(q)$ ($q = 0, \ldots, N_{\text{data}} - 1$) and each symbol-rate frequency hypothesis $k$ (selected from the bank in Step C), compute the $M_W$-way correlation on the carrier-compensated chip vector:

$$R_{\ell,k}[m] = \sum_{j=0}^{SF-1} \mathbf{u}_{\ell,k}[j] \cdot W_m[j], \quad m \in [0, 255]$$

Use a 1024-point Fast Walsh-Hadamard Transform (FHT) to compute the full correlation for $m \in [0, 1023]$, and then truncate to the first $M_W$ results for $m \in [0, 255]$. Select the hypothesis $k_{\text{best}}$ for this symbol using the peak energy criterion (Step C), and pass $R_{\ell,k_{\text{best}}}[m]$ to the soft-demapper.

**3. Soft Information (LLR) Generation**

Apply a decision-directed common phase rotation before soft-demapping to make the decided peak real-positive:

$$\widetilde{R}_\ell[m] = R_{\ell,k_{\text{best}}}[m] \cdot e^{-j \arg\left(R_{\ell,k_{\text{best}}}[\hat{m}_\ell]\right)}.$$

Let the metric be $D_\ell[m] = \Re\{\widetilde{R}_\ell[m]\}$ (or $D_\ell[m] = \Re\{R_{\ell,k_{\text{best}}}[m]\}$ if no additional rotation is used). For the $t$-th bit within each symbol ($t = 0, \ldots, k-1$), calculate the max-log LLR:

$$LLR_{qk+t} = \max_{m:\,((m \gg (k-1-t))\&1)=0} D_\ell[m] - \max_{m:\,((m \gg (k-1-t))\&1)=1} D_\ell[m]$$

The $N_{\text{data}} \cdot k = M$ LLRs correspond to the interleaved coded bits $B^{(\pi)}$. Apply the inverse permutation to deinterleave them before decoding:

$$LLR_i = LLR^{(\pi)}_{\pi^{-1}(i)}.$$

**4. CA-SCL Polar Decoding and Output**

Perform CRC-aided SCL (CA-SCL) Polar decoding on the deinterleaved LLRs to obtain the information bits $\hat{U}$. Use the existing CRC-32C (embedded inside $U$) as the list selection rule: choose the best-metric candidate that passes CRC; if none pass CRC, choose the best-metric candidate. Then perform a final CRC-32C check; if it passes, output $\hat{U}$ and truncate the payload according to the `Len` field in the Header. If it fails, discard the packet.